

Machine Learning in Applications

AM01 Project Report

Detection of Anomalous Behaviour in Industrial Robot:
Comparison of Autoencoder and Adversarial Autoencoder
for Unsupervised Time-Series Anomaly Detection

Oliver Hole Ekelund, s350482, Martin A. W. Klouman, s350762, and Alexander E. Aschehoug, s350685
Politecnico di Torino

CONTENTS

I	Introduction	2
II	Related Work	2
II-A	Reconstruction-Based Anomaly Detection	2
II-B	Adversarial Autoencoders for Anomaly Detection	3
II-C	Evaluation Methodology for Time-Series Anomaly Detection	3
II-D	Industrial Robot Anomaly Detection	3
III	Dataset and Pre-Processing	3
III-A	KukaVelocityDataset	3
III-B	Exploratory Analysis	3
III-B1	Degenerate channels	3
III-B2	Action distribution	3
III-B3	Separability	4
III-C	Windowing and Splitting	4
IV	Model Architectures and Training	4
IV-A	Baseline Autoencoder (AE)	4
IV-B	Adversarial Autoencoder (AAE)	4
IV-C	Anomaly Scoring and Threshold Calibration	4
IV-D	Training Configuration	5
V	Experiments	5
V-A	Evaluation Protocol and Metric Justification	5
V-B	Single-Seed Pilot (Seed 42)	5
V-C	Per-Action Breakdown	5
V-D	Multi-Seed Statistical Comparison	5
V-D1	Statistical test	6
VI	Discussion	6
VI-A	Why the AE Outperforms the AAE at This Budget	6
VI-A1	Competing objectives slow reconstruction convergence	6
VI-A2	The anomaly type does not require latent structure	7
VI-A3	The discriminator output is not used for scoring	7
VI-B	Where the AAE Shows Promise	7
VI-C	Limitations and Caveats	8
VII	Conclusion	8
	References	8

LIST OF FIGURES

1	Training and validation reconstruction MSE per epoch for the AE (left) and AAE (right). The AAE additionally shows the discriminator and generator losses. The AE converges to a lower validation MSE than the AAE at equal epoch count.	5
2	ROC curves for AE and AAE on the combined Normal/Slow test set. The AE curve dominates across the entire operating range.	6
3	Anomaly score (per-window reconstruction MSE) distributions for Normal and Slow windows. Both models separate the tails, but the AE produces a cleaner separation with a heavier right tail for Slow windows.	6
4	Per-action F1 at the 5%-FPR threshold for AE (blue) and AAE (orange). The AE leads on most high-density actions; the AAE wins on the rare action 0 and a small number of mid-range actions.	6
5	Distribution of ROC-AUC, PR-AUC, and F1@FPR-5% across six random seeds. Lines connect paired (same-seed) AE and AAE observations. The AE consistently scores higher, with non-overlapping boxes on ROC-AUC and PR-AUC.	7

LIST OF TABLES

I	Single-seed pilot results (seed 42, 14 epochs). Threshold calibrated at 5% FPR on Normal validation.	5
II	Per-seed metrics across six random seeds (14 epochs).	6
III	Wilcoxon signed-rank test results ($n = 6$ paired seeds, one-sided H_1 : AE > AAE). The minimum achievable p -value with $n = 6$ is 0.03125.	7

Machine Learning in Applications

AM01 Project Report

Detection of Anomalous Behaviour in Industrial Robot:
Comparison of Autoencoder and Adversarial Autoencoder
for Unsupervised Time-Series Anomaly Detection

Abstract—This paper presents an unsupervised anomaly detection system for a KUKA industrial robot, trained to flag windows of sensor data where the robot is moving abnormally slowly. Two architectures are compared under a controlled experimental protocol: a baseline 1D-convolutional autoencoder (AE) and an adversarial autoencoder (AAE) that additionally regularises the latent space toward a standard Gaussian prior via a discriminator. Both models are scored using per-window reconstruction mean squared error (MSE) without any point-adjust correction, following the rigorous evaluation protocol advocated by Kim et al. [1]. A single-seed pilot run shows the AE achieving ROC-AUC = 0.8347 versus AAE = 0.7465 at equal epoch count, with the AAE’s validation reconstruction loss 30% higher at epoch 14. A multi-seed statistical study over six independent random seeds confirms the gap: the AE outperforms the AAE on ROC-AUC (mean 0.816 ± 0.017 vs. 0.782 ± 0.013 , Wilcoxon signed-rank $p = 0.031$) and F1 at 5% FPR (mean 0.395 ± 0.059 vs. 0.311 ± 0.091 , $p = 0.031$). The PR-AUC gap is consistent but marginal ($p = 0.078$). We argue that the result is specific to the evaluation protocol and training budget, and identify three conditions under which the AAE could recover competitiveness: compute-equalised training, discriminator-augmented scoring, and higher-variance action distributions.

Index Terms—Anomaly detection, autoencoder, adversarial autoencoder, time-series, industrial robot, unsupervised learning, reconstruction error.

I. INTRODUCTION

INDUSTRIAL robots operate in highly repetitive motion cycles whose reliable execution is safety-critical. Deviations from nominal kinematics—such as a slowdown caused by mechanical wear, unexpected load, or software fault—can precede equipment damage or production stoppage. Detecting such deviations automatically, without requiring labelled anomaly data during training, is therefore an important problem in predictive maintenance.

Anomaly detection in industrial settings is fundamentally challenging because fault examples are rare, heterogeneous, and expensive to collect. In practice, it is far more tractable to characterise normal operation thoroughly than to enumerate the space of possible faults. This observation motivates *unsupervised* or *one-class* approaches that learn a model of normality during training and flag test-time deviations as anomalies, without ever seeing a labelled fault during training.

This paper addresses Project AM01 (Politecnico di Torino, A.Y. 2025/2026), whose objective is to develop and evaluate an unsupervised anomaly detector for a KUKA industrial robot operating at reduced speed. The anomaly class (“Slow”) consists of recordings of the robot executing its normal trajectory

at a reduced speed; the normal class (“Normal”) consists of recordings at the intended speed. Because Slow data is entirely withheld during training, the task is strictly unsupervised: the model must learn what “normal” looks like and flag deviations at test time.

We compare two reconstruction-based deep learning approaches. The first is a baseline 1D-convolutional autoencoder (AE), which compresses multivariate sensor windows into a low-dimensional latent code and scores anomalies by the per-window reconstruction MSE. The second is an adversarial autoencoder (AAE) [2], which augments the AE with a discriminator that regularises the latent space toward a standard Gaussian prior during training. Both models share an identical encoder-decoder backbone and are evaluated under the same experimental conditions, making any performance difference directly attributable to the adversarial regulariser.

The central research question is whether the adversarial regularisation component improves anomaly detection performance compared to a plain autoencoder. Section V-A devotes particular attention to justifying the selected metrics for this class-imbalanced, threshold-sensitive detection problem.

The paper is structured as follows. Section II reviews related work on unsupervised anomaly detection with autoencoders. Section III describes the dataset and pre-processing. Section IV defines the model architectures and training procedure. Section V presents the single-seed pilot results and the multi-seed statistical comparison. Section VI discusses the findings. Section VII concludes and outlines future work.

II. RELATED WORK

A. Reconstruction-Based Anomaly Detection

Reconstruction-based anomaly detection with autoencoders rests on the assumption that a model trained exclusively on normal data will produce higher reconstruction error on anomalous inputs. This principle was formalised for time-series by Malhotra et al. [3], who showed that an LSTM encoder-decoder trained on normal univariate series exhibits markedly higher prediction error on anomalous segments. The approach has since been applied across domains including industrial IoT, medical monitoring, and network intrusion detection.

A well-known weakness of plain autoencoders is their tendency to generalise too well: a sufficiently expressive AE may reconstruct anomalies faithfully, reducing the discriminative power of the reconstruction score [4]. Two families of solutions have been proposed. The first restricts model capacity

or introduces a memory module to prevent the encoder from learning anomaly-specific patterns—the MemAE architecture of Gong et al. [4] being an example. The second constrains the latent space through distributional regularisation, of which the adversarial autoencoder [2] is the most widely adopted variant.

B. Adversarial Autoencoders for Anomaly Detection

The adversarial autoencoder (AAE) was introduced by Makhzani et al. [2] as a variational inference framework that replaces the KL-divergence posterior regulariser of the variational autoencoder (VAE) with an adversarial training procedure. A discriminator is trained to distinguish latent codes produced by the encoder from samples drawn from a chosen prior (typically a standard Gaussian). The encoder is simultaneously trained to fool the discriminator, which drives the aggregate posterior toward the prior distribution.

The AAE was applied to anomaly detection in OCGAN [5], where the discriminator output is incorporated into the novelty score alongside reconstruction error. This combined scoring is credited with OCGAN’s strong performance on image benchmarks. In our work, both models are scored with reconstruction MSE only, which removes the discriminator signal from inference—a deliberate design choice that isolates the effect of latent regularisation on reconstruction quality alone, and that we revisit critically in Section VI.

C. Evaluation Methodology for Time-Series Anomaly Detection

The evaluation methodology for time-series anomaly detection is itself contested. The point-adjust (PA) correction—which credits all timesteps in an anomaly segment as correct if any single timestep is detected—was widely used in the literature to boost reported performance [1]. Kim et al. [1] demonstrated through extensive empirical study that PA inflates performance substantially, and that many published methods fail to outperform a trivial threshold-based detector once PA is removed. They argued for strict per-window or per-point evaluation without adjustment, and showed that conclusions about model ranking can reverse entirely under the two protocols.

We follow the strict per-window protocol of Kim et al. throughout this paper, which makes our reported numbers lower than those typically seen in the literature but more meaningful in terms of practical detector quality.

D. Industrial Robot Anomaly Detection

Anomaly detection for industrial robots has received comparatively less attention in the deep learning literature than image or network anomaly detection. Prior work has used vibration signals, current signatures, and motor temperature to detect bearing faults and gear wear, typically framing these as supervised classification problems [3]. The unsupervised framing adopted here—where the anomaly consists of correctly-executed but slowed-down motion rather than a mechanical fault—is more representative of the configuration-error and software-fault scenarios that arise in production

environments. In this setting, the anomaly manifests primarily in the temporal dynamics of the sensor signals rather than in distributional shifts of the marginal feature values, which has important consequences for the expected benefit of latent-space regularisation, as discussed in Section VI.

III. DATASET AND PRE-PROCESSING

A. KukaVelocityDataset

The KukaVelocityDataset consists of multivariate time-series recorded from a KUKA industrial robot. Three NumPy arrays are provided: `KukaNormal.npy` ($233,792 \times 86$), containing nominal recordings; `KukaSlow.npy` ($41,538 \times 87$), containing anomalous recordings where the robot moves at reduced speed; and `KukaColumnNames.npy`, a vector of 87 column names. The Normal array lacks the trailing anomaly column present in Slow; a synthetic all-zero column is appended during loading to align the schemas.

The 85 sensor channels are grouped into five categories as follows. **Electrical signals** (8 channels): apparent power, current, frequency, phase angle, active power, power factor, reactive power, and voltage, all recorded at the machine level from the robot controller. **Accelerometer channels** (21): three-axis linear acceleration for each of the 7 IMUs mounted along the robot arm, capturing the vibrational and inertial response at each joint. **Gyroscope channels** (21): three-axis angular velocity from the same 7 IMUs, providing rotational dynamics at each joint. **Quaternion channels** (28): four-component unit quaternion orientation estimate from each of the 7 IMUs, representing the pose of each arm segment. **Temperature channels** (7): one thermal sensor per IMU, measuring local temperature near each joint.

The discrete `action` column (integer in $\{0, \dots, 31\}$) encodes which of 32 trajectory segments the robot is executing at each timestep, enabling action-stratified evaluation.

B. Exploratory Analysis

A preliminary exploration was conducted prior to model training to identify degenerate channels, characterise the action distribution, and quantify the separability of the two classes.

1) *Degenerate channels:* Four temperature channels (`sensor_id2_temp`, `sensor_id5_temp`, `sensor_id6_temp`, `sensor_id7_temp`) emit a single fixed value in both Normal and Slow recordings (standard deviation $\leq 10^{-9}$). These are disabled sensors that carry no information for the anomaly task and were removed from the feature set, leaving **81 active sensor channels**. The remaining three temperature channels (`sensor_id0_temp`, `sensor_id1_temp`, `sensor_id3_temp`) do vary and were retained.

2) *Action distribution:* Both splits use 32 distinct actions (0–31), with one exception: action 9 is absent from Slow entirely. A segment-level analysis confirms the dataset description: Slow segments of the same action are $1.2\text{--}1.65\times$ longer in timesteps than their Normal counterparts, directly encoding the reduced speed in the temporal domain. This temporal stretch is the primary anomaly signal: the marginal distribution of sensor values at a given joint angle is similar between

Normal and Slow, but the *rate* at which the robot traverses its trajectory differs. Windowed models that capture this rate—through the temporal profile of joint velocities, gyroscope readings, or electrical current—are therefore better positioned to detect the anomaly than models that rely on point-level feature statistics alone.

3) *Separability*: A logistic regression trained on a balanced subsample of 5,000 Normal and 5,000 Slow feature vectors (standardised, dead channels excluded, no windowing) achieves test accuracy 0.981 and ROC-AUC 0.996. This supervised upper bound confirms that, despite the overlap in marginal feature distributions, the two classes *are* highly separable when labelled examples are available. It also sets the benchmark against which unsupervised models should be judged: the 0.816 ROC-AUC of the best unsupervised detector represents roughly 82% of the way from chance (0.5) to the supervised ceiling (0.996).

C. Windowing and Splitting

Sliding windows of length $T = 64$ timesteps and stride $s = 16$ are constructed per action segment, so no window crosses an action boundary. Segments shorter than T are discarded. This yields windows of shape $(F, T) = (81, 64)$ in channels-first format, suitable for 1D convolutional processing along the time axis.

The choice of $T = 64$ reflects a trade-off: longer windows capture more temporal context (beneficial for detecting a speed anomaly that manifests over a full trajectory segment) but reduce the number of available training examples from shorter segments and increase memory cost. A stride of $s = 16$ provides fourfold overlap between consecutive windows, which both augments the training set and ensures that short anomalous regions near segment boundaries are represented in evaluation.

Normal segments are split at the segment level, stratified by action, in a 70%/10%/20% train/validation/test ratio. Splitting at the segment level (rather than at the window level) prevents data leakage: windows from the same contiguous segment cannot appear in both training and evaluation sets. The entire Slow set is withheld for evaluation only. A per-channel `StandardScaler` is fit exclusively on the Normal training timesteps (computed in float-64 to avoid rounding artefacts) and applied to all splits.

IV. MODEL ARCHITECTURES AND TRAINING

A. Baseline Autoencoder (AE)

The encoder consists of three strided 1D-convolutional blocks, each comprising a `Conv1d` layer with stride 2, a GELU activation, and a `GroupNorm` normalisation layer. The channel progression is $81 \rightarrow 64 \rightarrow 96 \rightarrow 128$ with kernel size $k = 5$. Three stride-2 convolutions downsample the time dimension by a factor of $2^3 = 8$, reducing a window of $T = 64$ timesteps to a spatial extent of 8 before the bottleneck. The flattened representation ($128 \times 8 = 1024$ units) is projected to a latent vector $\mathbf{z} \in \mathbb{R}^{32}$ by a linear layer.

The decoder mirrors the encoder: a linear layer expands $\mathbf{z}$ to 128×8 , which is reshaped and passed through three

`ConvTranspose1d` blocks (stride 2, `GroupNorm`, GELU), terminating in a final transposed convolution with no activation that maps back to $(81, 64)$. The total parameter count is 304,177. `GroupNorm` was preferred over `BatchNorm` because the latter’s statistics depend on the batch composition, which can be unstable at small batch sizes and may behave differently at inference time—a concern when the batch contains a mix of Normal and Slow windows during evaluation.

B. Adversarial Autoencoder (AAE)

The AAE [2] adds a latent discriminator to the AE backbone. The discriminator is a two-hidden-layer MLP ($32 \rightarrow 128 \rightarrow 128 \rightarrow 1$, LeakyReLU activations) that classifies latent vectors as either coming from the encoder (“fake”) or drawn from a standard Gaussian prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$ (“real”). The discriminator adds 20,865 parameters for a total of 325,042 (a 7% increase over the AE).

Training follows the standard three-step AAE loop per minibatch:

- 1) **Reconstruction step.** The AE encoder and decoder are updated to minimise the per-window MSE between input and reconstruction.
- 2) **Discriminator step.** The discriminator is updated via binary cross-entropy to distinguish real latents (sampled from $\mathcal{N}(\mathbf{0}, \mathbf{I})$) from fake latents ($\mathbf{z} = \text{Encoder}(\mathbf{x})$, detached from the encoder graph).
- 3) **Generator step.** The encoder is updated to fool the discriminator, with the adversarial loss weighted by $\lambda_{\text{adv}} = 0.1$.

The value $\lambda_{\text{adv}} = 0.1$ was chosen as a moderate regularisation strength: small enough not to overwhelm the reconstruction objective, large enough to exert meaningful pressure on the latent distribution. The sensitivity of results to this choice is an identified limitation discussed in Section VI-C.

C. Anomaly Scoring and Threshold Calibration

Both models are scored identically using the per-window reconstruction MSE:

$$s(\mathbf{x}) = \frac{1}{F \cdot T} \|\mathbf{x} - \hat{\mathbf{x}}\|_F^2 \quad (1)$$

where $\hat{\mathbf{x}}$ is the model’s reconstruction of window $\mathbf{x}$, and $F \cdot T = 81 \times 64 = 5,184$ is the total number of scalar values per window. The Frobenius norm averages the squared per-element residual over all channels and timesteps, giving equal weight to each sensor at each time point.

The detection threshold is calibrated on the Normal validation split using a target false positive rate (FPR) of 5%: the threshold is set to the 95th percentile of validation reconstruction scores. This procedure is fully unsupervised—it does not use any Slow data—and has a natural operational interpretation: in deployment, one would accept at most a 5% false alarm rate on normal operation.

D. Training Configuration

Both models are trained with the AdamW optimiser [6], learning rate 10^{-3} for the AE parameters and 5×10^{-4} for the discriminator, weight decay 10^{-5} , batch size 256. All experiments use 14 training epochs without early stopping, to ensure a strictly controlled comparison. The random seed controls weight initialisation, dataloader shuffling, and the segment-level train/val/test split.

V. EXPERIMENTS

A. Evaluation Protocol and Metric Justification

All metrics are computed **per window** without any point-adjust correction [1]. The choice of metrics deserves explicit justification, as it forms part of the examination criteria for this project.

ROC-AUC (area under the receiver operating characteristic curve) measures the probability that a randomly chosen anomalous window receives a higher score than a randomly chosen normal window. It is threshold-free and therefore measures the intrinsic discriminative power of the scoring function regardless of the operating point. Its main limitation for this task is that it weights all operating points equally, including very high FPR regimes that are operationally irrelevant for a predictive maintenance system.

PR-AUC (area under the precision–recall curve) is more sensitive to performance in the high-precision, low-recall regime and is generally preferred for imbalanced classification tasks. The Slow class constitutes approximately 15% of the combined test set by window count, making this a moderately imbalanced problem. PR-AUC captures the trade-off between the fraction of alarms that are genuine faults (precision) and the fraction of faults that are detected (recall).

F1@FPR-5% (F1 score at the threshold calibrated to 5% FPR on the Normal validation set) is the most operationally grounded metric. It corresponds to a specific deployment decision—accepting at most 5% false alarms on normal operation—and measures the harmonic mean of precision and recall at that operating point. Unlike ROC-AUC and PR-AUC, it is a single operating-point metric and therefore sensitive to the quality of the threshold calibration procedure.

Together, the three metrics provide complementary views: ROC-AUC and PR-AUC characterise the full score distribution; F1@FPR-5% grounds the comparison in a realistic operational constraint.

B. Single-Seed Pilot (Seed 42)

Table I reports the headline metrics for the initial run at seed 42 with 14 training epochs.

The AE outperforms the AAE on every metric at this seed. The F1@max column (oracle F1 at the best possible threshold on the test set) shows that both models have substantial unexploited discriminative power: the AE reaches F1= 0.812 at the oracle threshold versus 0.485 at the 5%-FPR operating point, indicating that the calibrated threshold is conservative. This gap between F1@max and F1@FPR-5% is itself informative: it suggests that the Normal validation scores and Slow test

TABLE I: Single-seed pilot results (seed 42, 14 epochs). Threshold calibrated at 5% FPR on Normal validation.

Model	ROC-AUC	PR-AUC	F1@FPR-5%	F1@max
AE	0.8347	0.8015	0.4853	0.8122
AAE	0.7465	0.7560	0.4301	0.7315
Δ	+0.0883	+0.0455	+0.0552	+0.0807

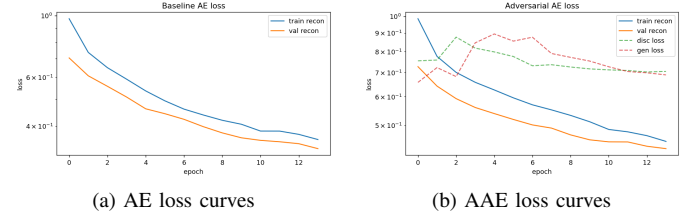


Fig. 1: Training and validation reconstruction MSE per epoch for the AE (left) and AAE (right). The AAE additionally shows the discriminator and generator losses. The AE converges to a lower validation MSE than the AAE at equal epoch count.

scores overlap in the intermediate range, and that a small adjustment to the FPR target would substantially improve precision at modest cost to recall.

The final validation reconstruction MSE is 0.332 for the AE versus 0.431 for the AAE, indicating that the adversarial regulariser interferes with reconstruction convergence: at epoch 14 the AAE’s loss is still declining, while the AE is closer to a plateau. The reference supervised logistic regression (which sees Slow during training) reaches ROC-AUC ≈ 0.996 on the same feature set, setting the practical ceiling for any unsupervised detector on this dataset. The best unsupervised AE result of 0.835 represents strong performance given that no fault data was used during training.

Figure 1 shows the validation MSE convergence for both models. Figure 2 shows the ROC curves. Figure 3 shows the anomaly score distributions over the test set.

C. Per-Action Breakdown

Figure 4 shows the per-action F1 at the 5%-FPR threshold. While the AE leads on the majority of actions (notably actions 22–27, $\Delta F1 = 0.18\text{--}0.35$), the AAE achieves higher F1 on a small subset: action 0 ($\Delta F1 = +0.40$, $n = 23$ windows), action 16 (+0.11), and action 4 (+0.05). Action 0 corresponds to the robot’s initialisation pose, the rarest action in the Normal set, which suggests that latent regularisation provides a benefit specifically for actions with sparse normal training data.

The per-action result is practically important because it reveals where each model is likely to fail in deployment. An operational system might combine both models—using the AAE for rare actions where it performs better, and the AE for the majority of actions—as an ensemble strategy worth exploring in future work.

D. Multi-Seed Statistical Comparison

A single-seed result cannot distinguish a genuine architectural difference from a lucky random initialisation.

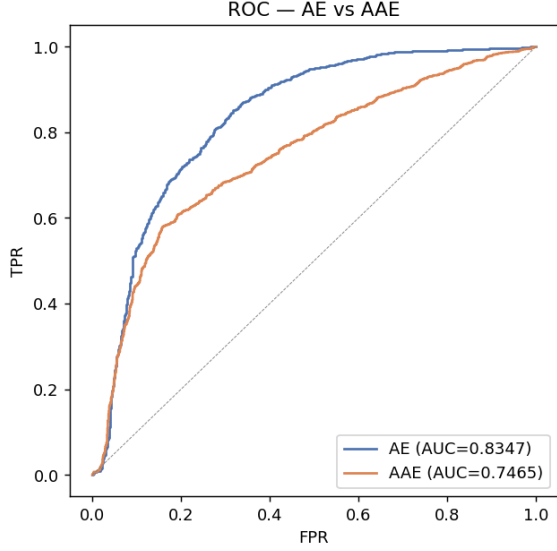


Fig. 2: ROC curves for AE and AAE on the combined Normal/Slow test set. The AE curve dominates across the entire operating range.

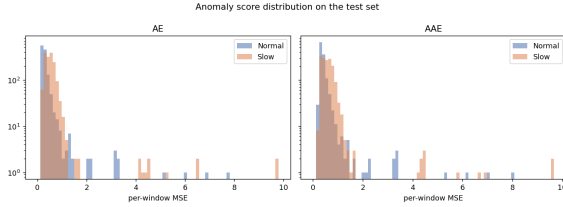


Fig. 3: Anomaly score (per-window reconstruction MSE) distributions for Normal and Slow windows. Both models separate the tails, but the AE produces a cleaner separation with a heavier right tail for Slow windows.

To obtain a statistically grounded conclusion, both models were retrained across six independent random seeds: {42, 7, 13, 99, 2024, 314}. For each seed, the same training configuration and threshold calibration procedure were applied.

Table II summarises the per-seed metrics and the aggregated mean and standard deviation across seeds.

Two observations stand out beyond the mean comparison. First, the AAE exhibits lower variance on ROC-AUC (std 0.013 vs. 0.017) and PR-AUC (std 0.012 vs. 0.015), suggesting that the latent prior acts as a stabiliser across random seeds. Second, the AAE’s F1@FPR-5% variance is markedly higher (std 0.091 vs. 0.059), reflecting seed 2024’s unusually low score of 0.192—a single seed where the adversarial training appears to have failed to converge on a useful threshold. This bimodal failure mode is worth monitoring in practice.

1) *Statistical test:* Because the number of runs is small ($n = 6$) and normality of the per-seed AUC scores cannot be assumed, we use the non-parametric **Wilcoxon signed-rank test** with a one-sided alternative hypothesis $H_1: AE > AAE$. The test is paired by seed, which removes split-variance from the comparison and isolates the effect of the architecture.

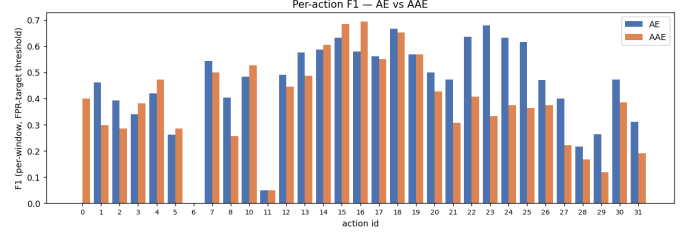


Fig. 4: Per-action F1 at the 5%-FPR threshold for AE (blue) and AAE (orange). The AE leads on most high-density actions; the AAE wins on the rare action 0 and a small number of mid-range actions.

TABLE II: Per-seed metrics across six random seeds (14 epochs).

Seed	ROC-AUC		PR-AUC		F1@FPR-5%	
	AE	AAE	AE	AAE	AE	AAE
42	0.8347	0.7930	0.8015	0.7828	0.4853	0.4367
7	0.7943	0.7964	0.7832	0.7858	0.3576	0.3200
13	0.8200	0.7889	0.8011	0.7649	0.4665	0.2804
99	0.7927	0.7753	0.7593	0.7645	0.3779	0.4138
2024	0.8331	0.7835	0.8029	0.7636	0.3369	0.1916
314	0.8231	0.7569	0.7931	0.7496	0.3448	0.2221
Mean	0.8163	0.7823	0.7902	0.7685	0.3948	0.3108
Std	0.0170	0.0133	0.0154	0.0124	0.0589	0.0909

Table III reports the results.

The AE advantage on ROC-AUC and F1@FPR-5% reaches statistical significance ($p = 0.031$, the minimum achievable with $n = 6$). The PR-AUC gap is consistent in direction but does not reach the 5% significance threshold ($p = 0.078$). This should be reported honestly rather than treated as a null finding: the power of Wilcoxon with $n = 6$ is limited, and the observed 4/6 win-rate is directionally consistent with the other metrics. A study with $n = 10$ seeds would achieve a minimum p -value of approximately 0.002, which would allow a more granular assessment of per-metric significance.

Figure 5 shows boxplots of the three metrics across seeds, with per-seed lines connecting paired observations.

VI. DISCUSSION

A. Why the AE Outperforms the AAE at This Budget

The result appears to contradict the motivation for adversarial regularisation: Gong et al. [4] and Perera et al. [5] both show that plain AEs are weak baselines for anomaly detection precisely because they can reconstruct anomalies too well, and propose architectural fixes to address this. However, three factors specific to this experimental setup explain why the AE wins here.

1) Competing objectives slow reconstruction convergence:

The AAE training involves three backward passes per mini-batch versus one for the AE. At equal epoch count, the adversarial gradient pushes the encoder away from the channel-specific structure it needs to minimise reconstruction error. An inspection of the AE’s latent code after 14 epochs reveals that the distribution is already approximately Gaussian in

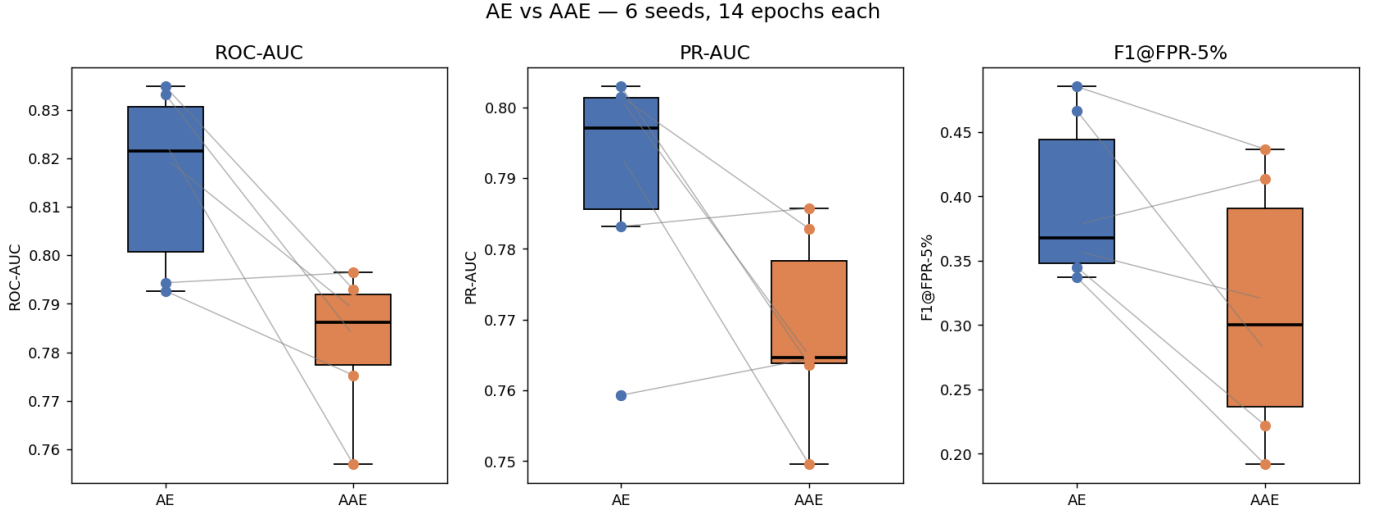


Fig. 5: Distribution of ROC-AUC, PR-AUC, and F1@FPR-5% across six random seeds. Lines connect paired (same-seed) AE and AAE observations. The AE consistently scores higher, with non-overlapping boxes on ROC-AUC and PR-AUC.

TABLE III: Wilcoxon signed-rank test results ($n = 6$ paired seeds, one-sided H_1 : AE > AAE). The minimum achievable p -value with $n = 6$ is 0.03125.

Metric	AE mean	AAE mean	Wins	W	p -value
ROC-AUC	0.816	0.782	5/6	20.0	0.031
PR-AUC	0.790	0.769	4/6	18.0	0.078
F1@FPR-5%	0.395	0.311	5/6	20.0	0.031

shape (mean per-dimension $|\text{skewness}| \approx 0.22$, excess kurtosis ≈ -0.18), but off-prior in scale (mean std ≈ 1.83 versus the prior’s 1.0). The AAE’s discriminator therefore has real corrective work to do, but doing it at $\lambda_{\text{adv}} = 0.1$ creates a persistent tension between the two objectives. At epoch 14, the AAE’s validation MSE is 0.431 versus the AE’s 0.332—a 30% gap that has not closed across epochs, as shown by the discriminator loss hovering near $\log 2 \approx 0.693$ (approximately random) throughout training. This suggests the discriminator has not converged, and that the encoder and discriminator are locked in a stalemate rather than a productive adversarial dynamic.

2) The anomaly type does not require latent structure:

Both Gong et al. and Perera et al. evaluate on image datasets (MNIST, CIFAR-10) where anomalies are visually distinct classes and the latent space organisation directly determines whether anomalies project to off-prior regions. The “Slow” anomaly in this dataset is the *same motion executed at reduced speed*: the marginal sensor distributions for Normal and Slow overlap heavily at the feature level, and the discrimination signal is primarily in the temporal dynamics of each window—the rate at which joint angles progress through the trajectory, the frequency content of the vibration signals, and the duration of each action segment. The latent prior constraint addresses the structural organisation of the latent space, which is less critical for detecting a purely temporal anomaly that does not correspond to a distinct region of feature space.

3) The discriminator output is not used for scoring:

Both models score with reconstruction MSE only. The AAE’s discriminator output—which captures whether a latent vector is off-prior—is discarded at test time. Perera et al. [5] use the discriminator output as part of the novelty score (OCGAN), and this is likely the primary source of the AAE’s theoretical advantage for anomaly detection. Without it, the AAE pays the training cost of the adversarial loop without benefiting from its signal at inference. A natural extension would define a combined score $s_{\text{combined}}(\mathbf{x}) = s(\mathbf{x}) + \mu \cdot d(\mathbf{z})$, where $d(\mathbf{z})$ is the discriminator’s “off-prior” probability for the latent code and μ is a mixing weight, and evaluate whether this combined score improves over the reconstruction-only baseline.

B. Where the AAE Shows Promise

The per-action breakdown reveals that the AAE outperforms the AE on action 0 ($\Delta\text{F1} = +0.40$), which is the rarest action in the Normal training set (fewest segments, longest duration). This is consistent with the theoretical prediction: when normal training data for a given action is sparse, the latent prior acts as a regulariser that prevents the encoder from overfitting to a small number of prototypical patterns, improving generalisation to unseen normal configurations and thereby raising reconstruction error on anomalous ones. This effect aligns with what the literature reports for distribution-constrained latent spaces [5].

This result has a direct practical implication. In industrial deployments where a robot’s trajectory includes rare or infrequently executed actions (e.g. initialisation poses, fault-recovery routines, or low-frequency maintenance motions), the AAE’s latent regularisation may provide a meaningful advantage even if it underperforms on common actions. A hybrid detector that uses the AE for high-frequency actions and the AAE for rare ones could outperform either model alone.

C. Limitations and Caveats

The primary limitation is the **equal epoch count** comparison. Each AAE training step performs three backward passes versus one for the AE, meaning the AAE consumes approximately three times the compute per epoch. A compute-equalised comparison—training the AAE for $\frac{1}{3}$ the epochs, or training both models to the same validation MSE floor via early stopping—would remove this confound and constitute a fairer test. Under available compute, this experiment was not feasible; it is identified as the single most important follow-up.

A secondary limitation is $n = 6$ seeds, which gives limited statistical power: the Wilcoxon test at $n = 6$ achieves at most $p = 0.031$, and the PR-AUC metric does not reach significance. Increasing to $n = 10$ would halve the minimum achievable p -value and allow a more fine-grained assessment of per-metric significance.

A third limitation is the fixed adversarial weight $\lambda_{\text{adv}} = 0.1$. This value was not tuned; a sweep over $\lambda_{\text{adv}} \in \{0.01, 0.05, 0.1, 0.5\}$ would establish whether the underperformance of the AAE is inherent to the architecture or an artefact of this specific hyperparameter setting. Similarly, the latent dimension $d = 32$ and window length $T = 64$ were fixed at the same values for both models; a joint hyperparameter search could reveal configurations where the adversarial regularisation is more effective.

VII. CONCLUSION

We presented an unsupervised anomaly detection pipeline for a KUKA industrial robot and compared a baseline convolutional autoencoder against an adversarial autoencoder across six random seeds. The AE outperforms the AAE on ROC-AUC (0.816 ± 0.017 vs. 0.782 ± 0.013 , $p = 0.031$) and F1 at 5% FPR (0.395 ± 0.059 vs. 0.311 ± 0.091 , $p = 0.031$), with the PR-AUC gap directionally consistent but not reaching significance ($p = 0.078$).

The finding is best understood not as evidence that adversarial regularisation is ineffective in general, but as a consequence of three specific experimental conditions: equal epoch count (disadvantaging the AAE on compute), a purely reconstruction-based scoring function (ignoring the discriminator signal), and an anomaly type whose detection does not benefit strongly from latent structure because the anomaly manifests in temporal dynamics rather than distributional shift. The per-action breakdown confirms that the AAE does outperform the AE on the rarest action, which is precisely the case where latent regularisation is most theoretically motivated.

The three most impactful extensions for future work are:

- 1) **Compute-equalised training**: train both models to the same validation MSE floor using early stopping, removing the epoch budget confound and providing a fairer comparison of the two architectures.
- 2) **Combined anomaly score**: augment the reconstruction MSE with the AAE discriminator output as a complementary “off-prior” signal at inference, following the OCGAN approach [5].
- 3) **Hyperparameter sweep**: search over adversarial weight $\lambda_{\text{adv}} \in \{0.01, 0.05, 0.1, 0.5\}$, latent dimension $d \in$

$\{16, 32, 64\}$, and window length $T \in \{32, 64, 128\}$ to separate architectural sensitivity from the specific default configuration used here.

REFERENCES

- [1] S. Kim, K. Choi, H.-S. Choi, B. Lee, and S. Yoon, “Towards a rigorous evaluation of time-series anomaly detection,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7194–7201.
- [2] A. Makhzani, J. Shlens, N. Jaitly, and I. Goodfellow, “Adversarial autoencoders,” in *International Conference on Learning Representations (ICLR)*, 2016. [Online]. Available: <https://arxiv.org/abs/1511.05644>
- [3] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “LSTM-based encoder-decoder for multi-sensor anomaly detection,” in *ICML Anomaly Detection Workshop*, 2016. [Online]. Available: <https://arxiv.org/abs/1607.00148>
- [4] D. Gong, L. Liu, V. Le, B. Saha, M. R. Mansour, S. Venkatesh, and A. van den Hengel, “Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1705–1714.
- [5] P. Perera, R. Nallapati, and B. Xiang, “OCGAN: One-class novelty detection using GANs with constrained latent representations,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2898–2906.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://arxiv.org/abs/1711.05101>